

Resumen patrones

Patrones estructurales

Adapter

- **Aplicabilidad:** Usar adapter cuando quiero usar una clase existente y la interfaz no es compatible con lo que preciso.
- **Ventajas**
 - Una misma clase Adapter puede usarse para muchos Adaptees (el Adaptee y todas sus subclases).
 - El Adapter puede agregar funcionalidad a los adaptados.
- **Desventajas**
 - Se generan más objetos intermediarios.

Decorator

- **Aplicabilidad**
 - Agregar responsabilidades a objetos individualmente y en forma transparente (sin afectar otros objetos).
 - Quitar responsabilidades dinámicamente.
 - Cuando subclasificar es impráctico.
- **Ventajas**
 - Permite mayor flexibilidad que la herencia.
 - Permite agregar funcionalidad dinámicamente.
- **Desventajas**
 - Mayor cantidad de objetos.
 - Complejo para depurar.

Proxy

- **Aplicabilidad:** Cuando se necesita una referencia más flexible hacia un objeto. Aplicaciones del proxy:
 - **Virtual proxy:** demorar la construcción de un objeto hasta que sea realmente necesario, cuando sea poco eficiente acceder al objeto real.
 - **Protection proxy:** Restringir el acceso a un objeto por seguridad.
 - **Remote proxy:** Representa un objeto remoto en el espacio de memoria local. Es la forma de implementar objetos distribuidos.

Estos proxies se ocupan de la comunicación con el objeto remoto, y de serializar/deserializar los mensajes y resultados.

- Para acceder a objetos que se encuentran en otro espacio de memoria, en una arquitectura distribuida.
- El proxy empaqueta el request, lo envía a través de la red al objeto real, espera la respuesta, desempaqueta la respuesta y retorna el resultado.
- En este contexto el proxy suele utilizarse con otro objeto que se encarga de encontrar la ubicación del objeto real. Este objeto se denomina Broker, del patrón de su mismo nombre.

- **Ventajas**

- Indirección en el acceso al objeto: El patrón Proxy introduce un nivel de indirección que permite controlar el acceso al objeto real, lo que ofrece flexibilidad en su uso.

- **Desventajas**

- Complejidad adicional.

Composite

- **Aplicabilidad:** Usar el patrón cuando:

- Quiera representar jerarquías.
- Quiera que los objetos “clientes” puedan ignorar las diferencias entre composiciones y objetos individuales. Los clientes tratan a los objetos simples y compuestos uniformemente.

- **Ventajas**

- Define “jerarquías” de objetos primitivos y compuestos.
- Los objetos primitivos pueden componerse en objetos complejos, los que a su vez pueden componerse recursivamente.
- Simplifica los objetos cliente. Los clientes usualmente no saben (y no deberían preocuparse) acerca de si están manejando un compuesto o un simple.
- Facilita agregar nuevos tipos de componentes porque los clientes no tienen que cambiar cuando aparecen nuevas clases componentes.

- **Desventajas**

- Puede resultar difícil proporcionar una interfaz común para clases cuya funcionalidad difiere demasiado. En algunos casos, tendrás que generalizar en exceso la interfaz componente, provocando que sea más difícil de comprender.

Patrones de creación

Builder

- **Aplicabilidad:** Usar el patrón cuando:
 - El algoritmo para crear un objeto complejo debiera ser independiente de las partes de que se compone dicho objeto y de cómo se ensamblan.
 - El proceso de construcción debe permitir diferentes representaciones del objeto que está siendo construido.
- **Ventajas**
 - Abstrae la construcción compleja de un objeto complejo.
 - Permite variar lo que se construye Director <-> Builder
 - Da control sobre los pasos de construcción
- **Desventajas**
 - Requiere diseñar e implementar varios roles.
 - Cada tipo de producto requiere un ConcreteBuilder.

Factory Method

- **Aplicabilidad:** Usar este patrón cuando:
 - Una clase no puede prever la clase de objetos que debe crear.
 - Una clase quiere que sean sus subclases quienes especifiquen los objetos que ésta crea.
 - Las clases delegan la responsabilidad en una de entre varias clases auxiliares, y queremos localizar qué subclase de auxiliar concreta es en la que se delega.
- **Ventajas**
 - Proporciona enganches para las subclases. Crear objetos dentro de una clase con un método de fabricación es siempre más flexible que hacerlo directamente.
 - Evita el acoplamiento entre el creador y los objetos concretos.
 - Aplica el principio OCP, permitiendo agregar nuevos tipos de productos sin modificar lo que ya funciona.
- **Desventajas**
 - Se puede complejizar el código si tenes que agregar muchos tipos de constructores.

Patrones de comportamiento

Strategy

- **Aplicabilidad:** Usar patrón cuando:
 - Existen muchos algoritmos para llevar a cabo una tarea.
 - No es deseable codificarlos todos en una clase y seleccionar cuál utilizar por medio de sentencias condicionales
 - Cada algoritmo utiliza información propia. Colocar esto en los clientes lleva a tener clases complejas y difíciles de mantener.
 - Es necesario cambiar el algoritmo en forma dinámica, en tiempo de ejecución.
- **Ventajas**
 - Mejor solución que subclasificar el contexto, cuando se necesita cambiar dinámicamente.
 - Desacopla al *Context* de los detalles de implementación de las *Strategy*.
 - Elimina condicionales.
- **Desventajas**
 - La clase cliente debe conocer las diferentes estrategias para poder elegir.
 - Overhead en la comunicación entre *Context* y *ConcreteStrategy*

State

- **Aplicabilidad:** Usar patrón cuando:
 - El comportamiento de un objeto depende del estado en el que se encuentre.
 - Los métodos tienen sentencias condicionales complejas que dependen del estado.
 - Este estado se representa usualmente por constantes enumerativas y en muchas operaciones aparece el mismo condicional. El patrón State reemplaza el condicional por clases **(es un uso inteligente del polimorfismo)**
- **Ventajas**
 - Localiza el comportamiento relacionado con cada estado.
 - Las transiciones entre estados son explícitas.
 - En el caso que los estados no tengan variables de instancia pueden ser compartidos.
- **Desventajas**
 - En general hay bastante acoplamiento entre las subclases de State porque la transición de estados se hace entre ellas, por lo que deben conocerse entre sí

Template Method

- **Aplicabilidad:** Usar patrón cuando:
 - Para implementar las partes invariantes de un algoritmo una vez y dejar que las subclases implementen los aspectos que varían.
 - Para evitar duplicación de código entre subclases.
 - Para controlar las extensiones que pueden hacer las subclases.
- **Ventajas**
 - Favorece el reuso de código.
- **Desventajas**
 - Algunas clases pueden verse limitadas por el template proporcionado.
 - Tienden a ser difíciles de mantener cuando el código escala.